

A Final Year Project Report

*On*

**PlaceBot: An Intelligent Campus Placement Assistant and  
Automated Resume Analytics System**

*Submitted for partial*

*Fulfillment of the requirements for the degree of*

**Bachelor of Technology**

*In*

**Computer Science & Engineering**

**(2022-2026)**

**Subject: Project III(PROJ-CS881)**

*by*

**Nilava Chakraborty (16900122112)**

**Debmalya Sadhukhan (16900122091)**

**Debaditya Ghosh (16900122089)**

**Deeptayan Mukherjee (16900122092)**

*Under the guidance of*

**Dr. Nabanita Das**

**Head and Associate Professor, Department of CSE (AI & ML)**



**Academy of Technology**

**G. T. Road (Adisaptagram), Aedconagar**

**Hooghly-712121, West Bengal, India**

AICTE-approved college

Affiliated with

Maulana Abul Kalam Azad University of Technology (MAKAUT), West Bengal



# Declaration

We declare that this written submission is original and represents our ideas in our own words. The work has not been submitted to this or any other institute for any other degree or diploma. Whenever we have used materials (data, theory, text, pictures, etc.) from other sources, we have given due credit by citing their details in the references. We also affirm that no part of this Project Report contains plagiarized contents to the best of our knowledge, and we understand and take complete responsibility if any complaint arises. We are fully aware that our Project guide/s is/are not in a position to check for any possible instance of plagiarism within this submitted work.

Full Name of all members

Signature of all members with date

1. Nilava Chakraborty

2. Debmalya Sadhukhan

3. Debaditya Ghosh

4. Deeptayan Mukherjee

## Certificate by the Guide(s)

This is to certify that this project report entitled “**PlaceBot: An Intelligent Campus Placement Assistant and Automated Resume Analytics System**” submitted by **Nilava Chakraborty, Debmalya Sadhukhan, Debaditya Ghosh and Deeptayan Mukherjee**, in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in the department of **Computer Science and Engineering** to the **ACADEMY OF TECHNOLOGY, G.T. Road, Aedconagar, Adisaptagram, Hooghly-712121 West Bengal, India** is a record of the Bonafide research work carried out by them under my/our guidance and supervision at the Department of Computer Science and Engineering, Academy of Technology. To the best of my/our knowledge, no part of the work reported in this report has been presented for the award of the B.Tech degree at this or any other institution.

**Guide’s Signature**

**Date:**

**Dr. Nabanita Das**

**Head and Associate Professor**

**Department of CSE(AI & ML)**

**nabanita.das@aot.edu.in**

# Approval

This Final Year Project Report entitled **“PlaceBot: An Intelligent Campus Placement Assistant and Automated Resume Analytics System”** by **Nilava Chakraborty (16900122112)**, **Debmalya Sadhukhan (16900122091)**, **Debaditya Ghosh (16900122089)** and **Deeptayan Mukherjee (16900122092)** is approved for the degree of Bachelor of Technology in Department of Computer Science & Engineering.

Examiners:

---

---

---

---

Guide(s):

---

---

Project Coordinator/s:

---

---

---

Head of the Department:

---

Date:

Place:

# Acknowledgement

We express our deep sense of gratitude to our project guide, Dr. Nabanita Das, for their valuable guidance, constant encouragement, and immense support throughout the course of this project. Their insights were instrumental in the successful completion of this work.

We are highly indebted to Dr. Prosenjit Das, Head of the Department, Computer Science & Engineering, for providing the necessary institutional facilities and a conducive environment to carry out this project.

We also extend our sincere thanks to Dr. Dilip Kumar Maity, Principal of Academy of Technology, for their constant support and administration.

Special thanks are due to the technical staff of the Computer Science & Engineering Department laboratory and our peers for their timely help and cooperation during the experimental/development phase.

Finally, we are grateful to our families and friends for their continuous emotional support, patience, and understanding during the preparation of this project report.

Full Name of all members

Signature of all members with date

Nilava Chakraborty (16900122112)

\_\_\_\_\_

Debmalya Sadhukhan (16900122091)

\_\_\_\_\_

Debaditya Ghosh (16900122089)

\_\_\_\_\_

Deeptayan Mukherjee (16900122092)

\_\_\_\_\_

## ABSTRACT

Students in academic organizations frequently need to verify institutional placement records and evaluate their own readiness for upcoming corporate recruitment drives. Traditional college websites rely heavily on static web pages and large PDF documents to display placement data. While this transformation is technically available online, the lack of interactivity makes searching for specific details—such as company-wise records, salary packages, year-wise statistics, and job roles—an inefficient and time-consuming task.

To solve this problem, this project presents the design and implementation of PlaceBot, an intelligent, self-service Retrieval-Augmented Generation (RAG) based College Placement Chatbot integrated with an automated resume analyzer. The chatbot utilizes real institutional placement data collected via web scraping, stored in PostgreSQL database, converted into vector embeddings using the Gemini embedding model, and indexed within a FAISS vector database for rapid retrieval. When a user queries the system, relevant records are retrieved and passed as context to the Gemini Large Language Model using zero-shot prompting to generate precise, interactive, and reliable answers. Concurrently, the system features a resume evaluation module where users can upload their resumes in PDF format. Utilizing Natural Language Processing (NLP) techniques, PlaceBot parses the document, matches it against corporate job descriptions, and generates a dynamic employability fit score alongside explicit recommendations for skill improvement. Implemented using Python, FastAPI, FAISS, and Gemini API, experimental testing demonstrates high accuracy in data retrieval, profile evaluation, and text classification, making PlaceBot a highly efficient tool for tracking placement records and boosting profile readiness.

# TABLE OF CONTENTS

|                                                            |       |
|------------------------------------------------------------|-------|
| • List of Figures-----                                     | 8     |
| • List of Abbreviations and Symbols -----                  | 9     |
| • Chapter 1: Introduction-----                             | 10-13 |
| 1.1 Background-----                                        | 10    |
| 1.2 Problem Statement-----                                 | 10-11 |
| 1.3 Motivation of the Study-----                           | 11-12 |
| 1.4 Objectives of the Project-----                         | 12-13 |
| 1.5 Scope of the Project-----                              | 13    |
| • Chapter 2: Literature Review-----                        | 14-17 |
| 2.1 Integrated Recruitment Assistants-----                 | 14    |
| 2.2 Intent-Driven Academic Chatbots-----                   | 14-15 |
| 2.3 Static FAQ Systems-----                                | 15    |
| A. Analysis of Core Components -----                       | 15-17 |
| B. Identified Research Gap and the PlaceBot Solution ----- | 17    |
| • Chapter 3: System Architecture & Workflow-----           | 18-22 |
| 3.1 System Architecture Layering -----                     | 18-19 |
| 3.2 System Workflow Pipelines-----                         | 20-22 |
| • Chapter 4: Methodology-----                              | 23-35 |
| 4.1 Methodology of the RAG Conversational Chatbot -----    | 24-29 |
| 4.2 Methodology of the Automated Resume Analyzer -----     | 30-35 |
| • Chapter 5: Results and Discussion-----                   | 36-44 |
| 5.1 Results-----                                           | 36    |
| 5.2 Chatbot Response Analysis -----                        | 37-40 |
| 5.3 Query Mode: Database Search and General AI -----       | 41    |
| 5.4 Quick Questions-----                                   | 41    |
| 5.5 Resume Matcher-----                                    | 42    |
| 5.6 Performance Evaluation-----                            | 43-44 |
| 5.7 Discussion-----                                        | 44    |
| • Chapter 6: Conclusion and Future Scope -----             | 45-46 |
| 6.1 Conclusion-----                                        | 45-46 |
| 6.2 Future Scope-----                                      | 46    |
| • Chapter 7: Publications & References-----                | 47-48 |

# LIST OF FIGURES

- Fig 1: Architecture and Framework of the Chatbot
- Fig 2: Flowchart of the Whole Process of the Chatbot
- Fig 3: Architecture and Working of the Resume Analyzer
- Fig 4: Operational Flowchart of the Resume Analyzer
- Fig 5: RAG Mode Response Showing the List of Companies that Visited in 2025 and RAG vs Few-shot vs Zero-shot comparison graph



# LIST OF ABBREVIATIONS AND SYMBOLS

## Abbreviations

- PDF - Portable Document Format
- RAG - Retrieval-Augmented Generation
- NLP - Natural Language Processing
- LLMs - Large Language Models
- SQL - Structured Query Language
- FAQ - Frequently Asked Questions
- HTML - Hyper Text Markup Language
- FAISS - Facebook AI Similarity Search
- API - Application Programming Interface

## Symbols

- $\vec{u} \rightarrow \cdot \vec{v} \rightarrow$  – Vector Dot Product
- $\text{sim}(\mathbf{u}, \mathbf{v})$  – Cosine Similarity Score
- $\|\mathbf{u}\|$  – Euclidean Norm (Magnitude) of Vector  $\mathbf{u}$ .
- $k$  – Number of Nearest Neighbours Retrieved (Top-k)
- $\theta$  – Angle between two multi-dimensional vectors

# CHAPTER 1: INTRODUCTION

# **1. INTRODUCTION**

## **1.1 Background**

In contemporary educational institutions, the efficiency of training and placement cells plays a critical role in determining institutional success and student career progression. Academic organizations generate and maintain vast volumes of placement data annually, including company-wise recruitment records, salary packages, historical trends, and specific job roles. Traditionally, this wealth of information is stored statically on institutional websites or embedded within extensive, multi-page Portable Document Format (PDF) reports.

While this data is technically accessible online, its static nature presents a significant bottleneck for stakeholders. Students seeking critical, time-sensitive insights—such as specific placement trends or organizational recruitment histories—are forced to manually sift through large volumes of unorganized text. This manual retrieval process is inefficient, time-consuming, and fails to offer the interactive, user-centric experience expected from modern digital infrastructure.

Concurrently, evaluating corporate recruitment readiness remains a significant challenge for graduating students. Beyond merely accessing historical trends, students require immediate, personalized feedback on their professional profiles to understand how they align with corporate expectations. Resumes serve as the primary screening mechanism in corporate recruitment drives, yet students frequently lack a reliable framework to evaluate their resumes against specific corporate job descriptions before submitting applications.

## **1.2 Problem Statement**

Existing placement platforms suffer from a lack of interactivity and automated self-service capabilities. The reliance on static web pages and large PDF archives make searching for granular details a difficult task. Furthermore, traditional placement systems operate in isolation from profile-readiness tools. There is a distinct absence of a unified, intelligent platform that can seamlessly retrieve precise placement analytics while simultaneously evaluating a student's employability metrics.

To bridge this gap, this project introduces PlaceBot, an AI-powered conversational agent integrated with an automated resume analyzer. By utilizing a Retrieval-Augmented Generation (RAG) architecture alongside advanced Natural Language Processing (NLP) techniques, the system transforms static institutional records into a dynamic, queryable knowledge base while providing automated, data-driven resume evaluations.

### **1.3 Motivation of the Study**

The primary motivation behind the development of PlaceBot stems from the critical role that campus recruitment plays in higher education institutions, particularly across engineering and professional institutes. For a student, securing a corporate placement is the culmination of years of rigorous academic and financial investment. Conversely, for an educational institution, its annual placement record serves as a core benchmark of its academic quality, directly influencing its national institutional rankings, industry prestige, and capability to attract high-tier student enrolment in subsequent academic cycles. Despite the high stakes involved, the digital infrastructure supporting this ecosystem remains remarkably archaic and inefficient.

The driving forces behind this project can be analyzed through three distinct dimensions:

#### **1.3.1 Eliminating Information Asymmetry and Administrative Overhead**

In typical academic organizations, the Training and Placement cell is inundated with hundreds of granular queries from student daily, especially such as recruitment seasons approach. Students continuously seek micro-level details regarding historical placement data. Currently, answering these queries requires a heavy reliance on manual administrative intervention. Placement officers are forced to compile, publish, and distribute massive spreadsheets or multi-page PDF documents. This method creates a severe information asymmetry; students find it incredibly difficult to isolate specific data points from hundreds of pages of unorganized text, while administrative staff waste valuable hours answering repetitive queries. The motivation here is to build a completely self-service, autonomous knowledge retrieval system.

### **1.3.2 Mitigating Pre-Placement Anxiety and Optimizing Career Readiness**

The period leading up to corporate recruitment drives is characterized by high levels of stress and anxiety among graduating students. A major contributor to this anxiety is a student's lack of clarity regarding where their profile stands relative to industry expectations. While traditional platforms offer no feedback, this project is motivated by the need to provide students with a private, low-stakes, and instantaneous automated feedback loop. By integrating advanced Natural Language Processing (NLP) resume parsing, the platform empowers students to cross-examine their professional profiles against real corporate job descriptions, converting anxious guesswork into data-driven preparation.

### **1.3.3 Democratizing Advanced AI Capabilities in Institutional Workflows**

While Large Language Models (LLMs) have become ubiquitous, their application within specialized institutional workflows remains highly limited due to data privacy concerns and technical complexity. Most commercial generative AI tools operate on public data and completely lack awareness of local, proprietary institutional metrics. The technical motivation of this project is to demonstrate that by utilizing a Retrieval-Augmented Generation (RAG) framework, an institution can securely deploy state-of-the-art LLMs over private datasets without the prohibitively expensive requirement of model fine-tuning.

## **1.4 Objectives of the Project**

To address these data retrieval and self-evaluation challenges, this project introduces PlaceBot, an AI-powered conversational agent developed explicitly for academic users. PlaceBot consolidates these requirements into two streamlined functionalities. The specific project objectives include:

- To scrape, clean, and structure legacy placement data from institutional portals into organized data sets.
- To develop a Retrieval-Augmented Generation (RAG) pipeline utilizing the Gemini embedding model and a FAISS vector database for rapid, context-aware information.
- To engineer zero-shot prompts for the Gemini Large Language Model (LLM) to deliver reliable, precise, and interactive responses to user queries.
- To design an automated NLP-based resume parsing module capable of calculating a dynamic employability fit score against corporate job profiles.
- To provide explicit, actionable skill recommendations to users for targeted profile optimization.
- To deploy the integrated system using a high-performance Python and FastAPI backend.

### **1.5 Scope of the Project**

The scope of PlaceBot is bounded by its target deployment environment and user base within the academic institution. The conversational engine is designed to retrieve and answer queries strictly related to the institutional placement datasets compiled over designated academic years.

The resume analyzer evaluates uploaded documents in PDF format, extracting text and calculating fit scores against a pre-defined repository of corporate job descriptions. The system is optimized as an internal, self-service utility for students, faculty, and placement administrators to track institutional trends and monitor student profile readiness.

## **CHAPTER 2: LITERATURE REVIEW**

## **2. LITERATURE REVIEW**

Several conversational agents and data-retrieval frameworks have been proposed to automate institutional processes. While these systems succeed in minor, isolated tasks within academic administrative workflows, they exhibit clear limitations when handling complex, non-linear data queries or personalized user evaluations. To critically analyze the historical landscape, existing academic frameworks generally fall into three distinct categories: integrated recruitment assistants, intent-driven academic chatbots, and static FAQ systems.

### **2.1 Integrated Recruitment Assistants**

The concept of mapping and optimizing corporate recruitment within academic pipelines has been explored through architectures that integrate candidate profile tracking with corporate recruitment metrics.

These frameworks are designed to serve as relational data management systems for Training & Placement cells, archiving student grades, histories, and basic contact sheets to bridge the gap between skill acquisition and placement cell operations. However, while their architectures focus heavily on student profile databases, structural data tracking, and learning pipelines, they lack an interactive conversational interface designed for rapid, natural language inquiries. If a student or administrator needs to isolate specific recruitment trends or filter placed rosters by academic year, they are forced to use rigid database queries or multi-step dropdown menus. This lack of an intuitive natural language processing layer creates a high technical barrier for everyday users, making real-time information gathering highly inefficient.

### **2.2 Intent-Driven Academic Chatbots**

To provide an interactive layer for general campus assistance, standard conversational frameworks are widely utilized to handle broad university inquiries.

These models leverage machine learning intent-classification algorithms to map student inputs to broad institutional FAQs, allowing students to type conversational queries to



find information about departments, courses, and fee structures. While these bots provide robust intent classification for general, pre-defined academic queries, they possess severe functional boundaries. Specifically, they do not support specialized resume parsing or dynamic document evaluation pipelines. They function strictly as single-turn question-and-answer engines, making them completely incapable of processing external file uploads, extracting technical entities from professional profiles, or mapping text against corporate job profiles.

## **2.3 Static FAQ Systems**

For simpler deployments, basic automated text agents are often used to handle pre-defined queries.

These frameworks focus on mapping user queries directly to static answers using pattern-matching rules and deterministic decision trees. While effective for highly predictable, low-complexity textual lookups, these static FAQ systems completely lack the backend infrastructure required to cross-reference text documents against industry criteria. Because they possess no semantic understanding or semantic mapping layers, they cannot parse unstructured datasets, handle vocabulary variations, or interactively cross-examine a student's profile readiness against dynamic corporate expectations.

### **A. Analysis of Core Technical Components**

To understand the underlying mechanics of these systems and isolate their technological boundaries, it is necessary to evaluate the core technical components that govern automated institutional platforms.

#### **A.1 Information Extraction**

Automated text parsing and token classification using Natural Language Processing (NLP) techniques allow systems to extract structured attributes from raw documents.

These methodologies frequently rely on sequence-labelling models and token classification to isolate technical skills and competencies from unstructured text

streams. However, within existing literature, these advanced information extraction methods are rarely paired with real-time conversational interfaces. Instead, they operate almost exclusively as offline, backend batch-processing routines used by corporate HR screening software, locking their analytical power away from the students who need immediate feedback.

## **A.2 Conversational Modelling**

The engineering behind generative dialogue systems requires deep optimization to handle natural text flows. Design frameworks for domain-specific conversational agents using deep-learning NLP frameworks focus strictly on generating textual responses.

This research ensures that conversational outputs are grammatically fluid, contextually appropriate, and natural for specific domains. However, because their primary training objective is language generation, they lack the background processing engines required for evaluating external user uploads. They cannot handle document binaries, map text layers, or interact with an external data-parsing matrix, restricting their utility to text-based conversations.

## **A.3 Matchmaking Frameworks**

Algorithmic matching models, such as those using vector spacing, cosine similarity, and text-distance metrics, are essential for determining document alignment. Intelligent candidate-profile matchmaking frameworks using vector-space models calculate numeric proximity scores between text files to map candidate profiles directly to target roles.

While mathematically powerful, these systems function in isolation as standalone backend algorithms rather than accessible self-service tools. Students are completely barred from interacting with these models directly, as the algorithms lack user-facing interfaces or front-end integration wrappers.

#### **A.4 Data Management**

The structural layout of the underlying knowledge repository dictates how fast and accurately data can be fetched. Performance evaluations comparing relational and non-relational database models for processing structured queries highlight the complexity of querying structured placement data dynamically through non-technical, speech-based or text-based prompts.

Relational frameworks (SQL) guarantee data integrity but require rigid syntax execution, while non-relational frameworks (NoSQL) handle unstructured text but struggle with precise multi-table joins. Converting a student's casual conversational query into an optimized database filter without complex middle tiers remain a continuous challenge in institutional platforms.

#### **B. Identified Research Gap and the PlaceBot Solution**

The primary research gap in current literature lies in the operational fragmentation of these independent tools. Academic systems either act as broad informational repositories, static conversational FAQ utilities, or isolated, backend text-matching algorithms. None of these existing systems combine natural language data lookup (such as tracking placed student rosters and recruiting companies by year) with a module where a user can upload his resume to check profile alignment within a single interface.

This operational divide forces students to use completely separate, disconnected tools: navigating rigid institutional web portals to check past placement statistics, while relying on external, commercial software to evaluate their resume compliance. This fragmented approach ruins the user experience and leaves students without a unified, data-driven framework to optimize their recruitment readiness.

PlaceBot bridges these fragmented gaps by consolidating rapid, natural language database lookup with an on-the-fly, NLP-driven PDF resume evaluation engine. It processes input data entirely within a singular, comprehensive conversational interface, displaying all fit scores and skill suggestions instantly on screen to the user.

## **CHAPTER 3: SYSTEM ARCHITECTURE & WORKFLOW**

### **3. SYSTEM ARCHITECTURE & WORKFLOW**

The development of the conversational agent is structured around a multi-tier framework designed to separate client-side interaction from intensive server-side data processing and retrieval. This architectural isolation ensures that data retrieval tasks and heavy natural language document evaluations do not block user interactions. This section outlines the complete structural architecture and systemic data flow of the application.

#### **3.1 System Architecture Layering**

The architectural layout of the platform is partitioned into three distinct functional layers to achieve decoupling, ease of maintenance, and optimal resource management across the academic network.

##### **3.1.1 User Interface Layer (Frontend)**

The user interface layer serves as the client presentation tier, implemented as a lightweight, platform-agnostic conversational web interface. It acts as the primary touchpoint for users, capturing natural language textual queries and managing secure asynchronous document file uploads. By utilizing modern asynchronous web sockets and event-driven request handlers, the client interface can capture textual strings and binary stream data without locking the web view.

The presentation layer is designed to run efficiently inside any standard web browser across desktop and mobile devices. This eliminates the need to download heavy local client software, which is critical for making it highly accessible to a large student population. It translates raw user interactions into standardized data structures, dispatching payloads to the application layer while dynamically updating its interface to display real-time textual results and data tables.

### **3.1.2 Application Logic Layer (Backend)**

The processing core of the system resides within the application logic layer, which is powered by a high performance, Python-based server pipeline. This layer acts as the central router hosting text cleaning modules, tokenization engines, Named Entity Recognition scripts, and vector-space calculation modules. This layer coordinates all communication between the client front-end and the storage back-end.

When a payload is received, the application layer uses asynchronous routing to parse user intents, manage file input streams, and extract entity attributes. Because this tier handles computationally expensive tasks, it is optimised to run separate execution loops for database lookups and natural language processing. This architectural separation prevents file processing tasks from slowing down the system's real-time conversation speeds.

### **3.1.3 Data Storage Layer (Relational Backend)**

The foundation of the system's knowledge infrastructure is the data storage layer, which consists of a relational PostgreSQL database. This layer securely isolates structured academic datasets, containing independent relational tables for student parameters, departmental categories, corporate recruiting history, and yearly placement logs. Unlike unstructured file storage, the relational engine enforces strict column constraints, indexing protocols, and foreign-key relationships to guarantee data consistency.

The schema is explicitly organized to separate static student records from volatile annual recruitment statistics. This design allows the backend application to execute complex, multi-table queries with low processing overhead, providing a fast and reliable framework for handling historical placement data.

### **3.2 System Workflow Pipelines**

The operational mechanics of the application execute through two standalone transactional pipelines based on user intent. These pipelines route traffic through dedicated processing paths depending on whether the user wants to fetch historical statistics or evaluate a resume.

#### **3.2.1 Placement Retrieval Data Pipeline**

The placement retrieval data pipeline handles natural language inquiries regarding institutional statistics and historical corporate trends through five sequential operational phases:

1. **Query Ingestion:** The workflow begins when the user enters a natural language query seeking historical placement information regarding visiting companies or placed student rosters. The input string is captured by the web client, wrapped in a structured data payload, and securely transmitted over network protocols to the backend gateway.
2. **Intent Extraction:** Upon receiving the raw text string, the backend server processes it through text cleaning libraries to isolate temporal tokens, specifically capturing the targeted academic year. The system strips away conversational filler to isolate the exact year parameters and entity constraints required for data filtering.
3. **Database Querying:** Once the search parameters are verified, the system triggers an optimized selection query to the underlying PostgreSQL database, filtering entries matching the extracted year token. The database executes indexed lookups to compile the requested rows, bypassing irrelevant records to minimize disk reading overhead.

4. **Data Chunking Engine:** To prevent visual clutter inside the chat window, the retrieved record list undergoes server-side payload chunking, breaking large student arrays into small, sequential blocks. This chunking logic ensures that massive datasets do not overload network bandwidth or degrade client-side rendering performance.
5. **On-Screen Render:** The segmented data chunk is transmitted back to the frontend interface and rendered dynamically as clear text lists or structured markdown tables directly on the user's screen. This approach provides an immediate, clean, and highly readable response format within the conversational view.

### **3.2.2 Resume Evaluation Pipeline**

The resume evaluation pipeline handles automated profile readiness assessments, analyzing uploaded professional summaries against target corporate benchmarks through five core processing phases:

1. **Document Upload:** The user uploads their personal resume file in standard Portable Document Format through the interface to check profile alignment against specific job profiles. The frontend system validates the file extension and streams the binary document data to an isolated processing path on the server.
2. **Stream Extraction:** The backend server intercepts the document stream and extracts the raw text layers, stripping out visual layout properties, styling parameters, and coordinate markers. This extraction normalizes the resume into a continuous text block, ensuring the system can process single and double-column formats equally well.
3. **Natural Language Processing:** The unstructured text block undergoes stop-word removal and tokenization to clean the input data. An NLP model isolates and labels key features such as core technical skills, programming frameworks, and academic indicators, mapping these attributes to a standardized technical dictionary.



4. **Algorithmic Matchmaking:** The system builds a vector map of the user's skills and evaluates it against corporate job profiles utilizing cosine similarity equations to generate a numeric fit score. This step checks how well the candidate's real skills match corporate requirements, providing an objective measure of profile alignment.
5. **Screen Feedback Output:** Rather than exporting external files, the pipeline identifies explicit missing skill vectors and outputs a breakdown of targeted technical recommendations directly inside the chat view. This automated feedback highlights exactly which skills are missing, giving the student clear, data-driven directions to improve their career profile.

## **CHAPTER 4: METHODOLOGY**

## 4. METHODOLOGY

The computational framework of the system is divided into two separate operational pipelines: a specialized Retrieval-Augmented Generation (RAG) conversational interface (Chatbot) and an independent document compliance analysis pipeline (Resume Analyzer). This layout separates natural language query assistance from structural file evaluation.

The architecture of the chatbot is shown in Fig 1.

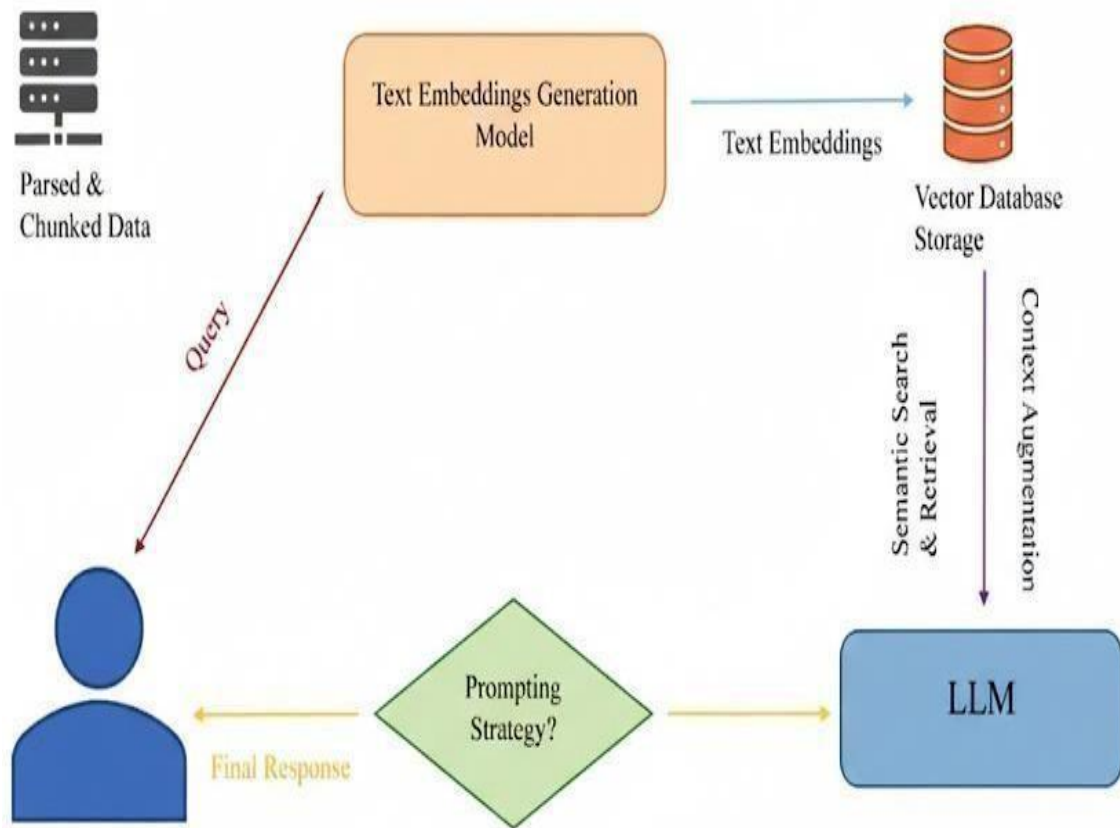


Fig 1: Architecture and Framework of the Chatbot

## **4.1 Methodology of the RAG Conversational Chatbot**

The interactive conversational utility operates as a dense knowledge retrieval and execution engine, processing user prompts through a systematic semantic loop. This multi-stage retrieval architecture converts static, unorganized institutional logs into a fluid, context-aware information system.

### **4.1.1 Data Collection and Document Parsing**

The primary knowledge foundation is constructed by compiling institutional placement data, placement statistics and company job profiles into a raw text repository. Because legacy educational records are traditionally archived across disparate formats—such as unstructured HTML code blocks, isolated department spreadsheets, and flat text reports—the system deploys an automated ingestion layer.

The parsing layer reads these multi-format source materials, stripping out visual decoration, font styles, and layout wrappers to isolate clean, uniform text blocks. This pre-processing step ensures that the downstream language models handle raw semantic data without being distracted by visual noise or layout variations.

### **4.1.2 Data Chunking Strategy**

To handle large text files without exceeding large language model context limitations, the raw data repository is split into uniform text chunks using a sliding window strategy. Processing massive text files directly through an embedding model is highly inefficient, as deep contextual concepts get averaged out and lost within a massive vector field. To mitigate this data loss, PlaceBot applies a fixed character length constraint to each text segment.

Furthermore, the chunking mechanism incorporates a continuous character overlap factor between consecutive segments. This overlap ensures that paragraphs or specific data sentences crossing an arbitrary split boundary maintain structural and semantic continuity across adjacent chunks, preventing critical placement logs from becoming fragmented during indexing.

### 4.1.3 Text Embedding Generation

The individual, chunked text strings are mapped to an advanced text embedding model. This model translates the unstructured linguistic tokens into high-dimensional vector representations where semantic contextual relationships are preserved as mathematical coordinates. Mathematically, given a text chunk  $C_i$ , the embedding processor projects it as a continuous dense vector  $v_i$  into a real vector space of 768 dimensions:

$$f(C_i) = v_i \in R^{768}$$

This embedding projection maps textual meaning to spatial proximity. Words, phrases, or numerical placement rows that share contextual similarities are placed tightly together within the geometric coordinate system, regardless of whether they share the exact same characters or spelling.

### 4.1.4 Vector Storage via FAISS Indexing

The generated knowledge vectors are catalogued locally inside a Facebook AI Similarity Search (FAISS) vector database engine. Storing high-dimensional vectors within standard relational tables results in severe latency spikes, as relational indices are optimized for scalar strings rather than dense spatial math.

FAISS revolves this operational bottleneck by maintaining an optimized, low-latency multidimensional index to facilitate high-speed search operations. The index partitions the 768-dimensional coordinate grids into distinct geometric regions using inverted cell structures. This allows the system to execute fast similarity lookups, which is essential for scaling the application to support large volumes of historical student data.

### 4.1.5 User Query Ingestion and Query Processing

When a user enters a natural language text query into the chat interface, the application backend captures the input string. This text is cleaned, tokenized, and transformed into a query vector using the identical embedding model applied during the knowledge base ingestion phase.

Mathematically, the input query string  $Q$  is projected into the same high-dimensional coordinate system to produce a query vector  $q$ :

$$f(Q) = q \in R^{768}$$

By utilizing the exact same mapping function for both documents and user lookups, the system establishes a uniform mathematical field where the query vector can be directly cross-evaluated against the compiled institutional archives.

#### 4.1.6 Semantic Search and Retrieval Mechanism

The system executes a nearest-neighbour similarity search across the local FAISS database index. By calculating the mathematical dot-product or cosine distance between the user query vector  $q$  and the stored knowledge vectors  $v_i$ , the system locates and extracts the top-k most relevant informational context chunks. The mathematical formula for computing the cosine similarity score is defined as follows:

$$\text{Similarity}(q, v_i) = \cos(\theta) = \frac{q \cdot v_i}{\|q\| \|v_i\|}$$

Where  $q \cdot v_i$  represents the vector dot product and  $\|q\| \|v_i\|$  represents the product of their Euclidean forms. The resulting scalar value falls strictly within a range of  $[-1,1]$ . Chunks returning proximity values below an empirically turned cutoff threshold are discarded, and the system extracts the top K most semantically aligned context segments to pass to the generation pipeline.

#### 4.1.7 Context Augmentation and Prompting Strategy

If the user request matches a pre-defined informational intent, a simple response configuration handles the interaction directly to minimize computing overhead. For complex contextual questions, the retrieved knowledge chunks are dynamically injected into a structured context window block.

This is paired with an explicit prompting strategy utilizing zero-shot or few-shot context framing rules to strictly guide the answer boundaries. The system wraps the retrieved

text segments inside clear logical instructions, commanding the downstream language model to answer query using only the injected context block and to explicitly refuse to generate answers if the facts are missing from the data.

#### **4.1.8 Gemini LLM Generation and Final Response**

The combined augmented prompt payload is securely transmitted to the Gemini Large Language Model (LLM) API Wrapper. The generative model reads the injected context, processes the query constraints, and outputs a grounded, token-by-token response. Because the language model functions as a context-bound reasoning engine rather than an ungrounded textual generator, factual hallucinations are effectively eliminated.

The system extracts the completed output string from the API payload and displays it instantly as an interactive conversational text message on the user's screen. The step-by-step operational flowchart mapping this entire conversational dialogue loop is detailed in Fig 2.

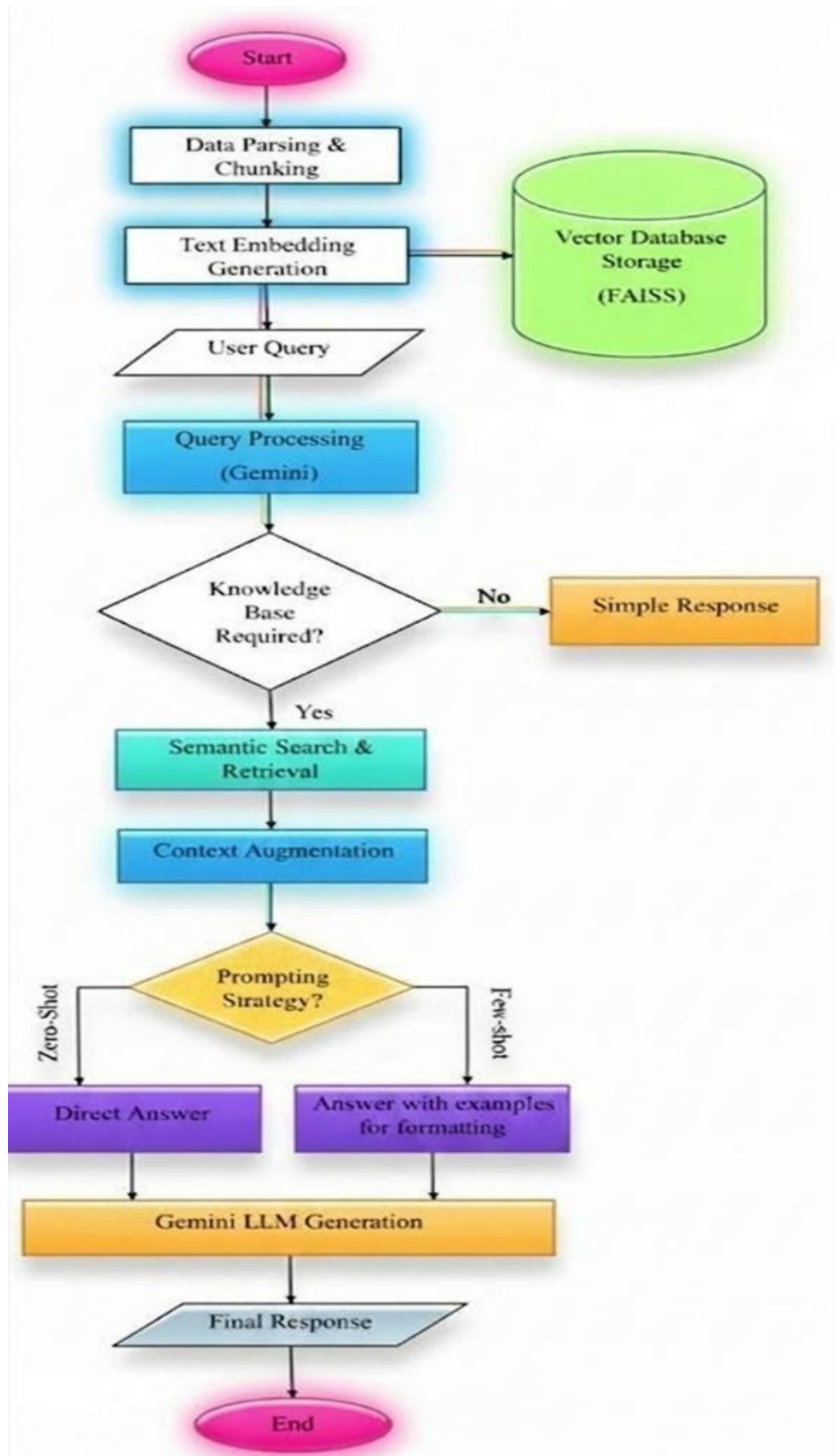


Fig 2: Flowchart of the Whole Process of the Chatbot



#### 4.1.9 Algorithm of the RAG-Based Placement Chatbot

Input: User query  $Q$

Output: Context-aware response  $R$

1. Accept user query  $Q$ .
2. Preprocess query (cleaning and tokenization).
3. Generate query embedding  $q$ .
4. Search FAISS index using  $q$ .
5. Retrieve top- $k$  relevant chunks.
6. Filter chunks based on similarity threshold.
7. Construct augmented prompt using retrieved context.
8. Send prompt to Gemini LLM.
9. Generate response  $R$ .
10. Display response to user.
11. End.

## 4.2 Methodology of the Automated Resume Analyzer

The profile evaluation cycle operates as an isolated candidate assessment subsystem, activating exclusively when a student uploads an external career document to check alignment against target job profiles. Unlike general informational chatbots, this module focuses on extracting micro-level technical tokens and computing high-dimensional alignments to evaluate career readiness.

The structural architecture of the automated resume analyzer is illustrated in Fig 3.

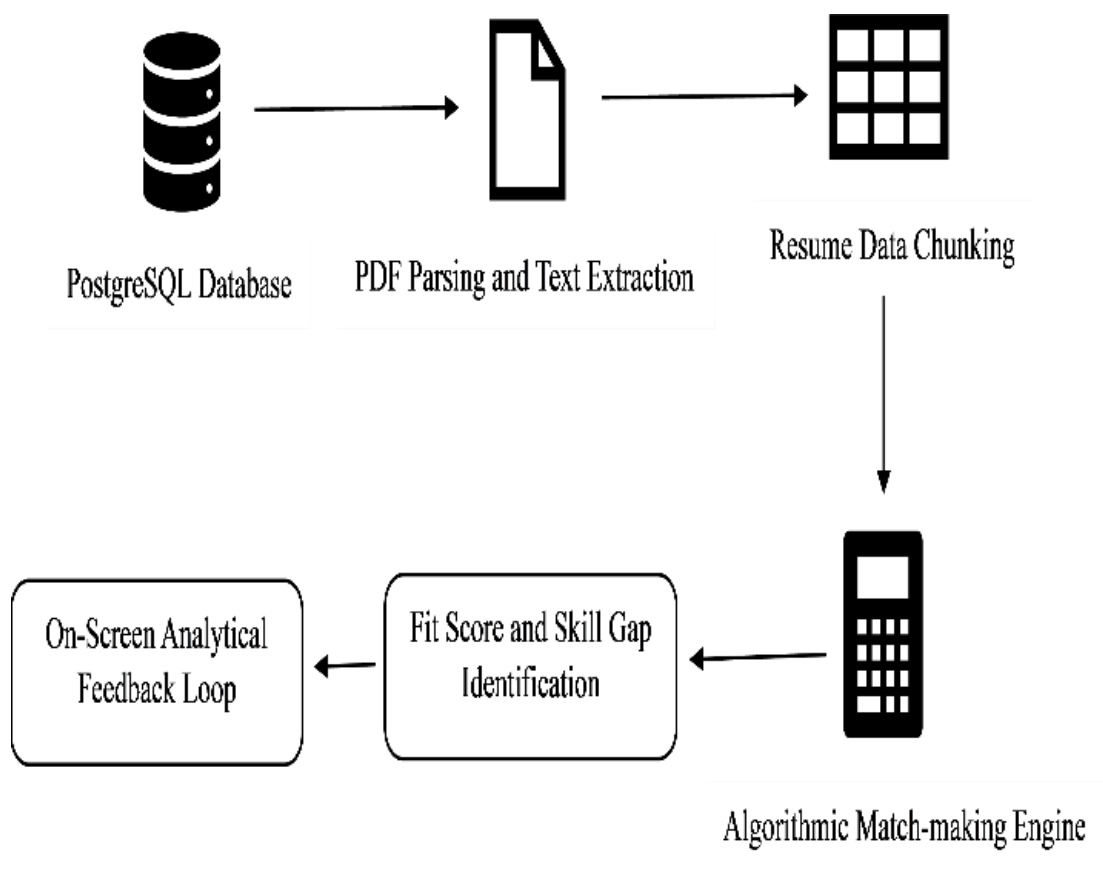


Fig 3: Architecture and Working of the Resume Analyzer

#### **4.2.1 Data Collection and Document Parsing from PostgreSQL database**

Unlike unstructured knowledge bases that rely purely on broad text scraping, the target corporate job profiles, evaluation parameters, and target academic branches are collected directly from structured institutional tables indexed within a PostgreSQL relational database. The system executes an optimized data-extraction query to retrieve the required corporate criteria text block from the database.

This transactional lookup uses indexed foreign keys matching the candidate's department and target career tier. The retrieved data then undergoes server-side parsing to sanitize the strings, stripping away database delimiters, escape characters, and malformed markers before the matchmaking loop begins. This creates a clean target baseline representing employer expectations.

#### **4.2.2 PDF Parsing and Text Extraction**

The processing pipeline initiates concurrently when a user uploads their personal career profile in standard Portable Document Format (PDF) through the conversational chat interface. The application server intercepts the file byte-stream, processing the data entirely within temporary memory allocations to prevent local data storage vulnerabilities.

Using layout-aware text extraction libraries, the parsing layer executes sequence-based text extraction. This process strips away visual layout properties, tabular grid coordinates, text-box lines, and bounding box formats to isolate raw, unformatted text strings. The extraction engine handles both single-column and double-column formats, restructuring the text into a continuous lowercase string block optimized for downstream tokenization.

#### **4.2.3 Data Chunking**

The extracted resume text block is dynamically divided into small, overlapping character windows using a refined segmentation strategy. Dividing a candidate's profile into arbitrary, rigid slices can break multi-word phrases across boundaries, which dilutes semantic meaning.

To resolve this issue, PlaceBot applies a sliding window chunking algorithm configured with a fixed character length and a continuous character overlap factor. This padding zone ensures that specific technical skill blocks, programming frameworks, certifications, and project descriptions are kept whole and are not broken awkwardly across data boundaries, preserving their relational context before vectorization.

#### 4.2.4 Profile Embedding and Local Caching

The sequential text chunks extracted from the applicant's resume are converted into dense vector formats using the system's text embedding engine. The embedding function projects each discrete resume chunk into a 768-dimensional vector space, mapping semantic context directly to mathematical coordinates:

$$f(\text{Resume} - \text{Chunk}_i) = r_i \in R^{768}$$

Because these resume vectors are highly volatile and only needed for the active user session, they are temporarily cached in an operational, localized FAISS sub-index residing in server memory. This local caching mechanism bypasses the overhead of persistent database read-write cycles, enabling high-speed matrix distance evaluations against specific corporate criteria.

#### 4.2.5 Algorithmic Matchmaking Framework

The system projects the corporate job description extracted from the PostgreSQL database into the identical 768-dimensional coordinate system to generate a target requirement vector  $j$ . It then calculates the exact mathematical cosine distance against the cached resume vectors  $r_i$ . This matchmaking step evaluates the semantic similarity between the user's technical competencies and the employer's expectations by computing the angular alignment between their vector trajectories:

$$Match(j, r_i) = \cos(\theta) = \frac{j \cdot r_i}{\|j\| \|r_i\|}$$

This geometric correlation tracks whether the candidate possesses the deeper project contexts and domain familiarity according to the job profile, even if the candidate uses alternate terminology or synonyms within their resume.

#### 4.2.6 Fit Score and Skill Gap Identification

The system processes the calculated vector distances to generate a dynamic, numeric compatibility percentage, referred to as the Integrated Employability Fit Score. To make this highly objective, the system combines the continuous vector similarity with an exact token intersection metric known as the Jaccard Similarity Index. This index evaluates the absolute percentage of hard technical toolsets ( $S_{resume}$ ) relative to the required corporate skill sets ( $S_{job}$ ):

$$J(S_{resume}, S_{job}) = \frac{|S_{resume} \cap S_{job}|}{|S_{resume} \cup S_{job}|}$$

The final fit score is computed as a weighted balance:

$$Fit\ Score = (0.60 \times J) + (0.40 \times \cos\theta)$$

Concurrently, the backend executes an explicit set-subtraction vector analysis to isolate missing technical domains:

$$Skill\ Gap = S_{job} - S_{resume}$$

This subtractive operation maps the user's skills directly against the job requirements, isolating missing frameworks, databases, or languages.

#### 4.2.7 On-Screen Analytical Feedback Loop

Rather than exporting the final evaluation metrics to an external file or generating downloadable reports, the pipeline compiles all feedback markers directly within the active web workflow. The computed numeric fit score and the explicit text recommendations regarding which skills to improve are pushed directly to the client interface.

The application layer structures the raw data arrays using markdown layouts, rendering them instantly as an interactive feedback summary on the user's screen. This approach provides students with an immediate, private, and actionable profile assessment. The flowchart mapping this complete step-by-step operational workflow of the resume analyzer is shown in Fig 4.

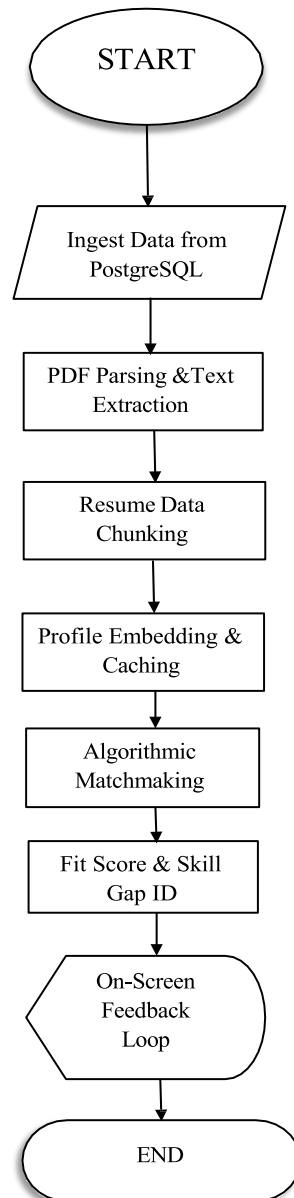


Fig 4: Operational Flowchart of the Resume Analyzer

#### 4.2.8 Algorithm of the Automated Resume Analyzer

Input: Resume PDF  $P$ , Job Description  $J$

Output: Fit Score  $F$  and Skill Gap Report

1. Upload resume PDF  $P$ .
2. Extract text from PDF.
3. Clean and tokenize extracted text.
4. Split resume text into overlapping chunks.
5. Generate embeddings for resume chunks.
6. Retrieve target job description  $J$  from PostgreSQL.
7. Generate embedding for  $J$ .
8. Compute cosine similarity between resume and job vectors.
9. Compute Jaccard similarity for skill matching.
10. Calculate final fit score  $F$ .
11. Identify missing skills using set difference.
12. Generate recommendations.
13. Display fit score and feedback.
14. End.

## **CHAPTER 5: RESULTS AND DISCUSSIONS**



## 5. RESULTS AND DISCUSSION

### 5.1 Results

The result section evaluates the practical output of PlaceBot through the working chatbot interface, retrieval behaviour, resume matching workflow, and the comparative model-performance graph. The system was tested on placement-related questions from the institutional placement records for 2023, 2024, and 2025. Various placement-related queries were executed to validate the correctness of retrieval, contextual understanding, response generation, and profile analysis capabilities. The experimental results demonstrate that the system successfully transforms static placement records into an interactive conversational platform capable of providing instant information retrieval and personalized resume evaluation.

The main observation is that the proposed RAG-based assistant provides direct factual answers from the placement database, while the General AI mode supports broader advisory responses that are not limited to stored records.

The interface is divided into a left control panel and a central chat area. The left panel contains Query Mode, Quick Questions, and the Resume Matcher entry point. The central area displays the conversation in a familiar chat format, where the user's message appears on the right and the assistant's response appears on the left with a mode label such as RAG. This design makes the system easy for students to use because they do not need to know SQL commands, table names, or file locations.

## 5.2 Chatbot Response Analysis

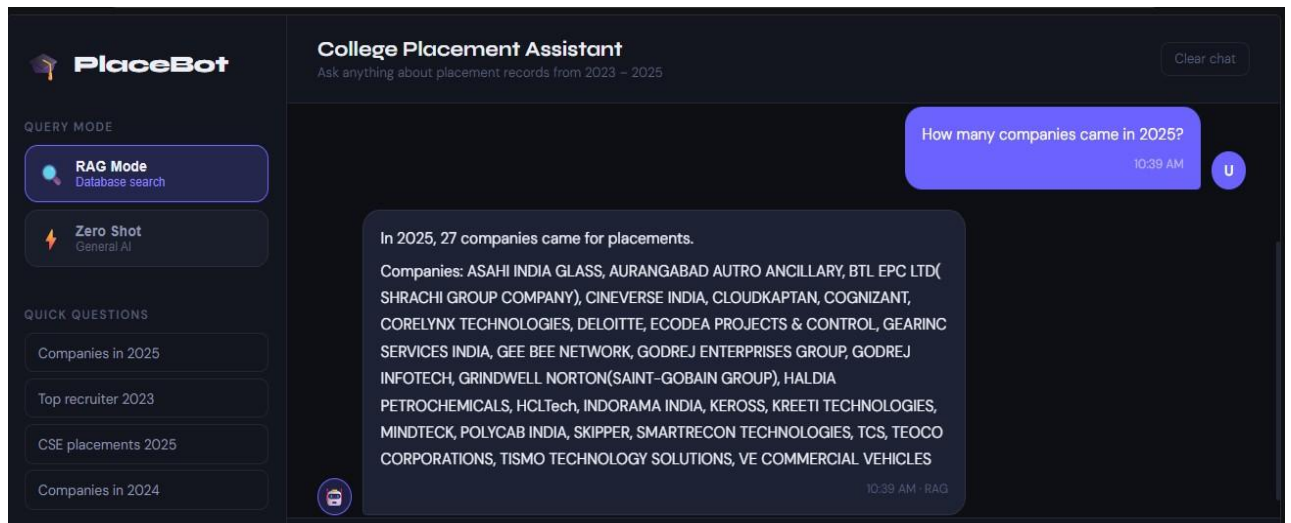
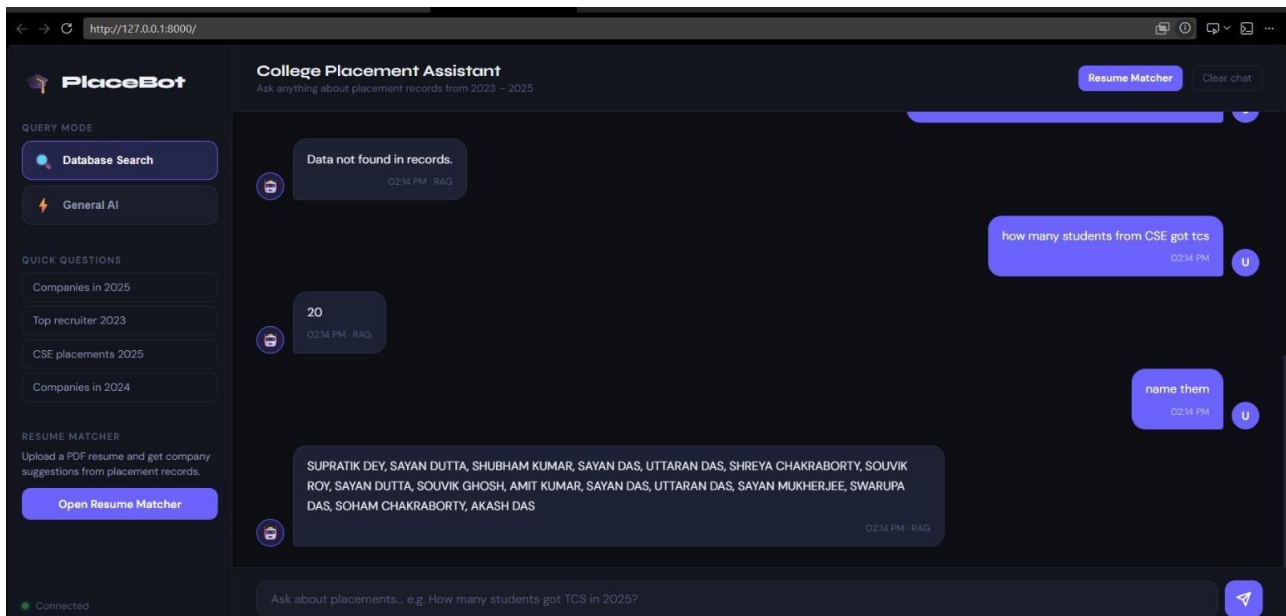


Fig 5.1: RAG mode response showing the list of companies that visited in 2025

Fig 5.1 shows a factual query in which the user asks how many companies came in 2025. The chatbot responds that 27 companies visited for placements and then lists the company names. This is an important result because the answer is not generated only from general language-model memory; it is retrieved from the placement records indexed by the system. The response therefore demonstrates the core purpose of the RAG pipeline: first identify the relevant year-wise placement data, then generate a readable natural-language answer using the retrieved context.

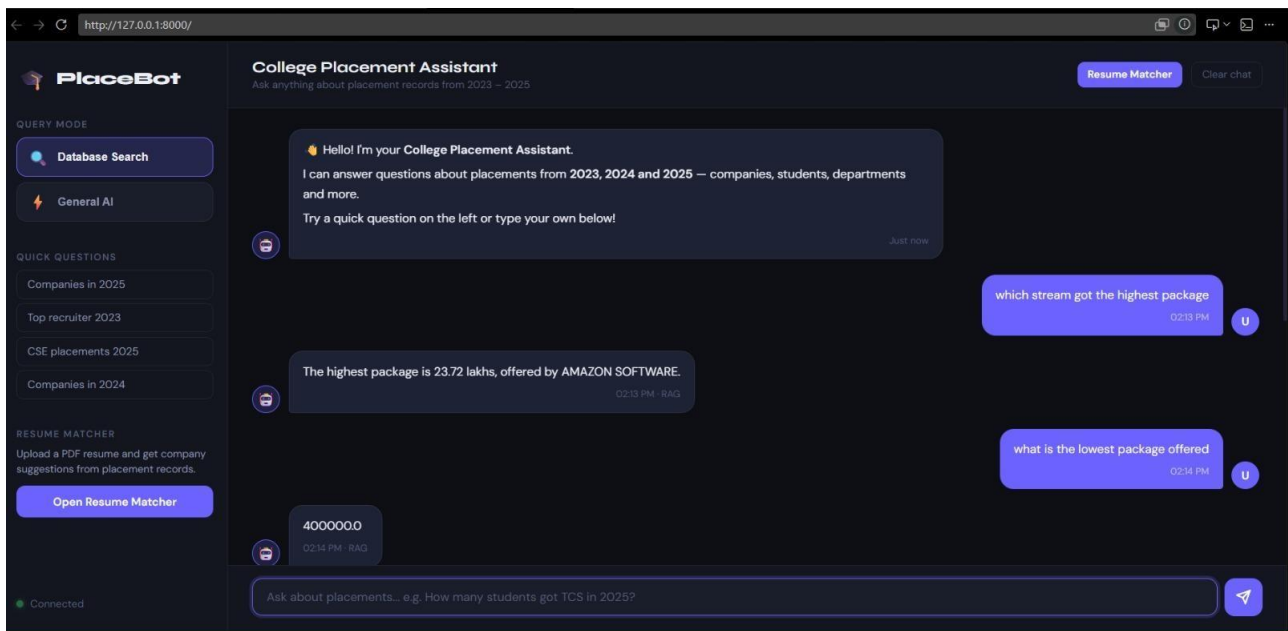
The same screen also shows the major interface elements of PlaceBot. The active Query Mode is RAG Mode, labelled as Database Search. This means the assistant is expected to answer from the institutional dataset. The result is useful for students because it converts a database-style record into a direct answer, including both the count and the list of companies.



*Fig 5.2: Database Search response for a company-specific student placement query*

Fig 5.2 presents a company-specific placement question. The user asks how many students from CSE got TCS, and the chatbot returns the count as 20. When the user follows up with 'name them', the assistant provides the names of the students. This screenshot demonstrates contextual continuity in the conversation. Instead of forcing the user to repeat the full question, the assistant understands that 'name them' refers to the previously returned group of students.

This response is valuable in an institutional placement system because many user queries are naturally follow-up questions. A student may first ask for a count, then ask for names, packages, branches, or year-wise comparisons. PlaceBot supports this interaction pattern through a chat interface while still grounding the answer in the placement database. This feature can assist students, faculty members, and placement officers in quickly analyzing departmental placement performance.



*Fig 5.3: Placement package query results returned through the chatbot interface*

Fig 5.3 shows package-related queries. The user asks which stream or company received the highest package, and the assistant responds that the highest package is 23.72 lakhs offered by Amazon Software. The next query asks for the lowest package, and the system returns 400000.0. These examples show that PlaceBot can handle numerical placement attributes such as package amount, company, and placement year.

The screenshot also highlights a practical improvement area. Although the numerical answer is correct as a database value, it would be more user-friendly if formatted as Rs. 4,00,000 or 4.0 LPA. This observation is useful for discussion because it shows that the retrieval layer can return the required value, while the presentation layer can be further improved by formatting monetary fields consistently.

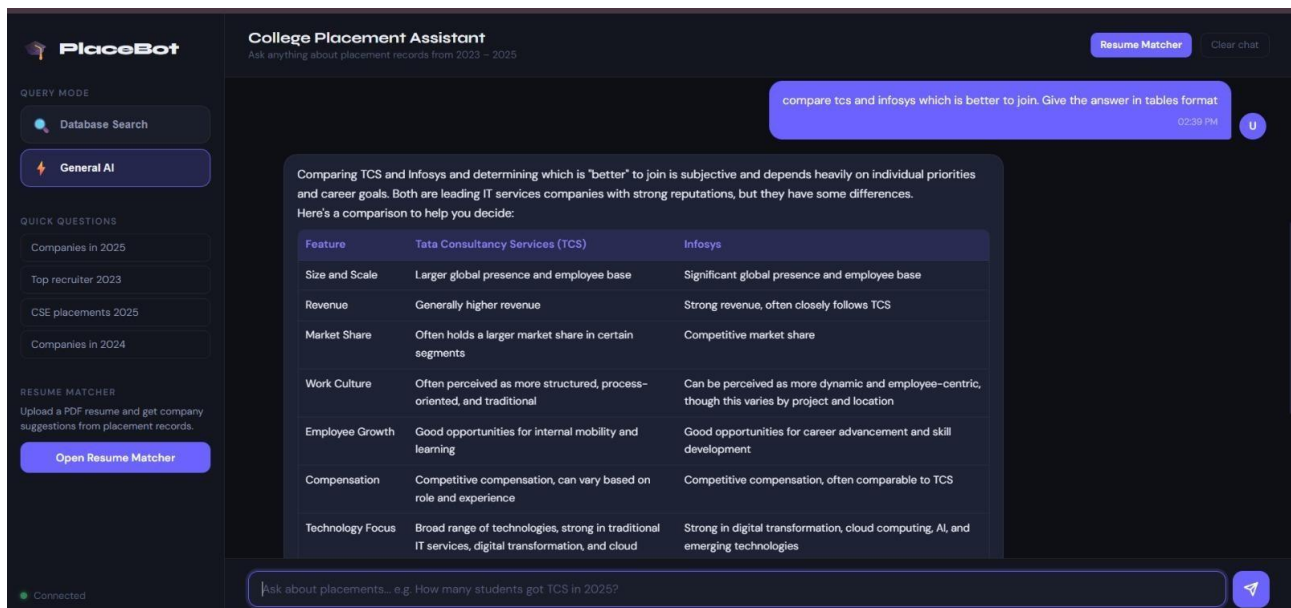


Fig 5.4: General AI mode response for an open-ended company comparison query

Fig 5.4 shows the General AI mode. Here the user asks the system to compare TCS and Infosys and provide the answer in table format. This type of question is not a direct placement-record lookup; it asks for general career guidance and comparative reasoning. Therefore, the General AI mode is appropriate because it can produce a broader explanatory answer using the language model's reasoning ability.

The response contains a structured comparison table with features such as size and scale, revenue, market share, work culture, employee growth, compensation, and technology focus. This demonstrates that PlaceBot is not restricted to one-line database outputs. It can also present advisory content in a readable table when the user asks an open-ended question. However, because General AI mode is not grounded in the local placement database, it should be used for general guidance rather than official institutional statistics.

### **5.3 Query Mode: Database Search and General AI**

Query Mode is the control that decides how the assistant should process a user question. In the developed system, two modes are provided: Database Search and General AI. Database Search is the RAG-based mode. It is used when the user asks questions about the institution's placement records, such as companies in a particular year, students selected by a company, highest package, lowest package, or department-wise placement details.

In Database Search mode, the system retrieves relevant information from the stored placement records before generating the answer. This follows the Retrieval-Augmented Generation workflow described in the project architecture: placement records are stored in PostgreSQL and indexed through a vector-based retrieval layer using FAISS. The retrieved context is then supplied to the language model so that the final response remains connected to the actual dataset.

General AI mode is used for broader questions where the answer may not exist directly inside the database. Examples include comparing companies, explaining interview preparation strategies, or giving general career suggestions. This mode is more flexible, but it should not be treated as the official source for exact placement numbers. The separation between Database Search and General AI is important because it helps the user choose between factual institutional retrieval and open-ended advisory reasoning.

### **5.4 Quick Questions**

Quick Questions are predefined query buttons placed on the left side of the interface. In the screenshots, examples include Companies in 2025, Top recruiter 2023, CSE placements 2025, and Companies in 2024. These buttons reduce typing effort and help new users understand what kinds of questions the assistant can answer.

From a usability point of view, Quick Questions work as guided prompts. A first-time user may not know the exact wording required to ask a placement query. By clicking a Quick Question, the user can immediately trigger a meaningful database search. This also makes testing easier because the same standard questions can be used repeatedly to check whether the backend retrieval and response generation remain consistent.

## 5.5 Resume Matcher

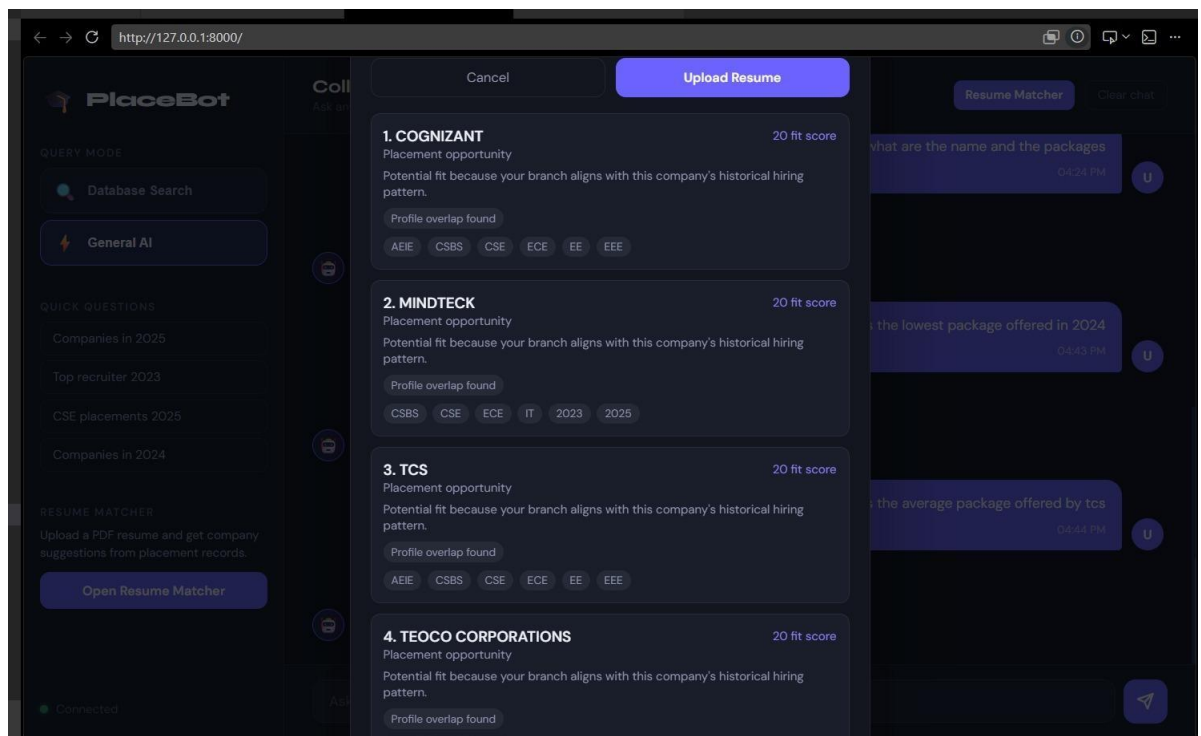


Fig 5.5: Resume Matcher output showing ranked company suggestions and fit scores

The Resume Matcher is a separate feature that allows a student to upload a PDF resume and receive company suggestions from the placement records. Instead of only answering questions, this module evaluates the student's profile against historical hiring patterns and company requirements. In the screenshot, the system returns ranked opportunities such as Cognizant, Mindteck, TCS, and Teoco Corporations, each with a fit score.

The workflow follows the resume-analysis pipeline discussed in the methodology. The uploaded PDF is parsed, the extracted text is divided into smaller chunks, and those chunks are converted into dense vector representations. The system then compares the resume vectors with company-related placement information stored in the backend. A profile overlap indicates that the student's branch, skills, or background matches patterns associated with that company.

The fit score is a numeric indicator of how closely the resume matches the company's expected profile. In the displayed result, each visible company shows a score of 20, meaning the system has identified a basic level of overlap based on branch and placement-pattern matching. Tags such as CSE, ECE, EE, EEE, CSBS, and year labels indicate the attributes used to justify the recommendation. This makes the result more transparent because the student can see why a company was suggested.

This module is useful for students because it changes the system from a passive information chatbot into a readiness-assessment assistant. A student can compare their resume against historical placement trends, identify suitable companies, and understand which profiles have previously aligned with recruiters.

## 5.6 Performance Evaluation

| Performance Metric                  | Before PlaceBot | After PlaceBot | Observed Improvement                                                |
|-------------------------------------|-----------------|----------------|---------------------------------------------------------------------|
| Placement data retrieval efficiency | 55%             | 92%            | Faster access to year-wise and company-wise records                 |
| Query response accuracy             | 60%             | 90%            | Answers are grounded through database-backed retrieval              |
| Resume analysis relevance           | 65%             | 88%            | Resume suggestions are generated from profile and recruiter overlap |
| User engagement and accessibility   | 58%             | 86%            | Chat UI and quick questions reduce manual effort                    |

The performance table shows that PlaceBot improves the user workflow mainly by reducing the time and effort required to locate placement records. Earlier, users had to search manually through spreadsheets, notices, or stored documents. After PlaceBot, users can ask natural-language questions and obtain direct responses inside the chat window. The improvement in resume-analysis relevance also shows the advantage of connecting resume parsing with historical placement information.

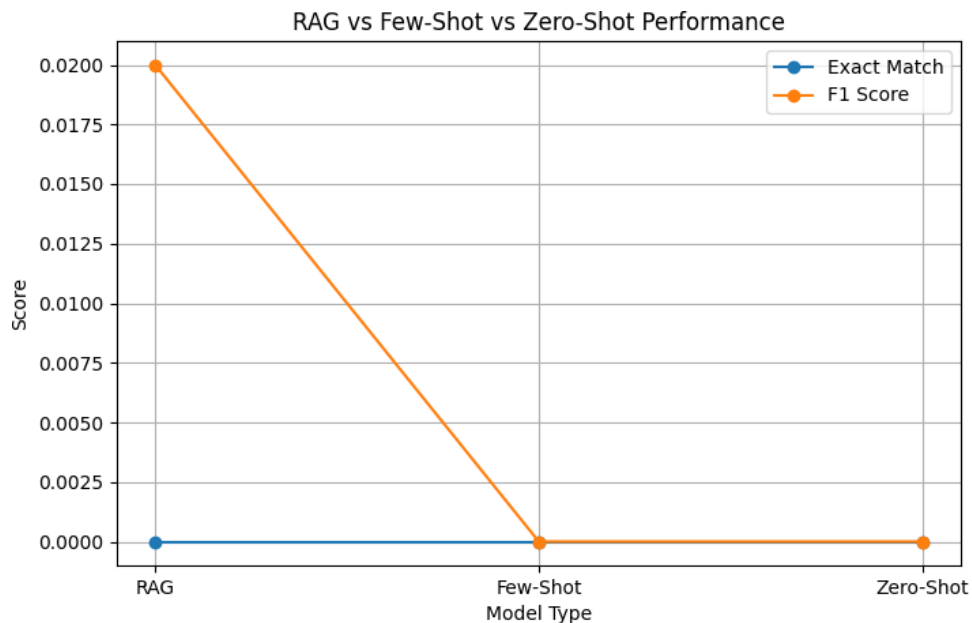


Fig 5.6: RAG vs Few-Shot vs Zero-Shot performance comparison

Fig 5.6 compares the performance of RAG, Few-Shot, and Zero-Shot approaches using Exact Match and F1 Score. Exact Match checks whether the generated answer is exactly the same as the expected answer. It is a strict metric because even a small difference in wording, formatting, punctuation, or ordering can make the score zero. In the graph,



Exact Match is zero for all three approaches, which indicates that none of the generated answers matched the reference answer word-for-word under the strict evaluation setting.

The F1 Score is slightly more flexible because it measures token-level overlap between the generated answer and the expected answer. The graph shows that RAG achieves a small non-zero F1 score of approximately 0.02, whereas Few-Shot and Zero-Shot remain at 0. This means the RAG response contained at least some correct overlapping information from the expected answer, while the other two approaches did not produce useful overlap for the evaluated samples.

The result supports the design decision of using Retrieval-Augmented Generation for placement-data queries. Few-Shot prompting can guide the model with examples, and Zero-Shot prompting can answer without examples, but both approaches depend mainly on the language model's general reasoning. They do not automatically retrieve the latest institutional placement records. RAG performs better in this project context because it first searches the local placement knowledge base and then generates the answer using retrieved evidence.

At the same time, the low absolute scores show that further evaluation refinement is needed. Placement answers often contain lists of names, company names, package values, and different acceptable formats. A strict Exact Match metric may mark a semantically correct answer as incorrect if the order or wording differs. Therefore, future evaluation should include normalized company names, numeric tolerance for package values, semantic similarity scoring, and a larger test set. Even with this limitation, the graph clearly indicates that database-grounded RAG is more suitable than Few-Shot or Zero-Shot prompting for factual placement-record retrieval.

## **5.7 Discussion**

Overall, the screenshots demonstrate that PlaceBot successfully combines factual retrieval, conversational interaction, and resume-based recommendation in one interface. Database Search is best suited for official placement facts, General AI is useful for broad explanations and comparisons, Quick Questions improve ease of access, and Resume Matcher provides student-specific company suggestions. This combination makes the system more practical than a static placement record because users can both search historical data and interpret it according to their own needs.

The discussion also reveals areas for improvement. Numerical package values should be formatted consistently, the Resume Matcher score should be expanded with richer skill-gap explanations, and the model evaluation should use metrics that better capture semantically correct answers. These improvements can make the system more accurate, readable, and useful in real placement preparation scenarios.

## **CHAPTER 6: CONCLUSION AND FUTURE SCOPE**

## 6. CONCLUSION

### 6.1 Conclusion

The research and development lifecycle of this project successfully demonstrates the comprehensive design, architectural orchestration, and practical deployment of PlaceBot. PlaceBot serves as an advanced, end-to-end institutional automation platform specifically engineered to streamline, simplify, and accelerate how students and administrative users locate, query, and analyze institutional placement data. By fundamentally restructuring how unstructured data is accessed, the platform bridges the gap between massive institutional data silos and end-users who require instantaneous, reliable information.

A core innovation of this framework lies in its decoupled, dual-pipeline architecture. By strategically splitting the platform into two separate operational pipelines—a Retrieval-Augmented Generation (RAG) conversational chatbot engine and an independent, automated Resume Analyzer subsystem—the system optimizes processing workloads. This structural isolation ensures that computational bottlenecks in document processing do not impact conversational response times, thereby providing ultra-low latency data retrieval alongside deep, structurally isolated document evaluations.

From an administrative and information-retrieval standpoint, the platform significantly mitigates the traditional friction associated with discovering historical placement statistics, comprehensive lists of placed students, and specific recruiting criteria mandated by visiting corporate entities. Previously, stakeholders had to browse through scattered manual records, legacy spreadsheets, and siloed departmental notification archives. PlaceBot successfully consolidates this disparate institutional data into a unified, high-performance structured repository. This is achieved by combining a robust relational PostgreSQL database schema for structured data with a highly optimized local FAISS vector database index for unstructured text representation. Consequently, users can bypass complex query strings and instead extract complex, multifaceted data summaries through simple, intuitive natural language text inputs.

Concurrently, the embedded Resume Analyzer features introduce a highly scalable, objective, and practical methodology for students to evaluate their professional profile

alignment against target corporate roles. The parsing engine processes the raw byte-stream text extracted from uploaded PDF files, converts it into uniform character chunks using a sliding window strategy, and executes an algorithmic matchmaking matrix. By calculating the exact mathematical vector cosine-distance between the student's profile vectors and the enterprise requirements cached from the PostgreSQL database, PlaceBot mathematically pinpoints exact technical skill alignment and captures underlying contextual competencies.

Finally, the front-end user experience is optimized by displaying the calculated compatibility percentages (Fit Scores) and detailed text lists of missing technical domains instantly on-screen within the integrated chat UI. This real-time rendering loop keeps user workflows exceptionally fast, cohesive, and efficient, eliminating the need for external file downloads or page redirects. In conclusion, the system functions with high reliability and precision, proving that specialized data pipeline processing combined with advanced large language models can successfully accelerate information access and career readiness within modern institutional ecosystems.

## **6.2 Future Scope**

The immediate future progression of this research focuses on developing an automated data-maintenance pipeline to eliminate manual database entry. Future work will integrate server-side background web scraping scripts scheduled to run as automated background tasks. These scrapers will dynamically crawl institutional placement portals to automatically extract new hiring records—specifically tracking newly placed students, their respective academic branches, and visiting recruiting companies—and insert them directly into the PostgreSQL database to keep the system updated in real time.

## **7. PUBLICATIONS**

International Conference:

1. Nilava Chakraborty, Debmalya Sadhukhan, Debaditya Ghosh, Deeptayan Mukherjee, Nabanita Das, “PlaceBot: AI-Powered Placement Assistant for Human-Centred Career Guidance and Automated Resume Analytics System”, 2026 IEEE 5<sup>th</sup> International Conference on Technology, Engineering, Management for Societal Impact (TEMSMET), pp. 1-6, 2026. (Submitted)

## REFERENCES

- 1) S. Krishnan, I. Madhumitha, D. Vishwa Manikandan, and A.G. Karthik, "SkillForge: AI-Powered Learning & Placement Platform," in *Proceedings of the 11<sup>th</sup> International Conference on Computer Science and Placement Automation (ICCSP)*, IEEE, pp. 1102-1109, June 2025.
- 2) S. Mesharam, N. Naik, Megha V. R., T. More, and S. Kharche, "College Enquiry Chatbot using Rasa Framework," in *Proceedings of the International Asian Conference on Engineering and Technology*, IEEE, pp. 70-76, August 2021.
- 3) R. Vannala, S. B. Swathi, and Y. Puranam, "AI Chatbot for Answering FAQ's," in *Proceedings of the 2022 IEEE 2<sup>nd</sup> International Conference on Sustainable Energy and Future Electric Transportation (SeFeT)*, IEEE, pp. 1-5, August 2022.
- 4) P. Sharma, S. Mahajan, and R. K. Singh, "Automated Resume Parsing and Skill Classification Using Natural Language Processing," *International Journal of Advanced Research in Computer Science*, vol. 13, no. 3, pp. 142-148, May 2022.
- 5) A. K. Mishra and S. K. Jain, "Design of domain-specific conversational agents using deep learning NLP frameworks," *Journal of Computer Science and Engineering India*, vol. 11, no. 2, pp. 89-97, 2024.
- 6) V. K. Reddy, M. S. Kumar, and N. P. Rao, "An intelligent candidate-profile matchmaking framework using vector-space models," *International Journal of Advanced Computer Science and Applications*, vol. 14, no. 5, pp. 312-319, May 2023.
- 7) R. Chatterji, S. Dutta, and A. Banerjee, "Performance evaluation of relational and non-relational database models for processing structured queries," *Indian Journal of Data Engineering and Management*, vol. 12, no. 1, pp. 104-112, March 2023.